



Ariel Soto

Escuela de Administración y Negocios
Universidad de Concepción · Campus Chillán

SEMESTRE 1 · 2026



Fundamentos de Programación para Análisis Económico

MÓDULO II · SEMANA 5 · SESIÓN 1

Los Verbos de dplyr

Hoy cobramos la promesa

Toda la **Semana 4** dijimos: “esto se hace más limpio con *dplyr*”. Hoy es ese día.

El **tidyverse** es una familia de paquetes con una gramática común para trabajar datos. **dplyr** es su verbo: la manipulación de tablas.

Al terminar esta sesión podrás:

- ▶ Usar los cinco verbos centrales: `select`, `filter`, `mutate`, `summarise`
- ▶ Explicar cada uno frente a su equivalente en **R base** (Semana 4)
- ▶ **Encadenar** operaciones con el pipe `|>` y leer un análisis como una frase

💬 **Activación:** lo que la semana pasada costaba tres líneas con `$` y corchetes, hoy cabe en una que se lee casi en español.

Instalar y cargar: no es lo mismo

```
install.packages("tidyverse")    # UNA sola vez en la vida del computador  
library(tidyverse)              # al inicio de CADA script
```

Instalar descarga el paquete (como comprar una app). **Cargar** lo activa en la sesión (como abrir la app). dplyr viene dentro de tidyverse: no se carga aparte.

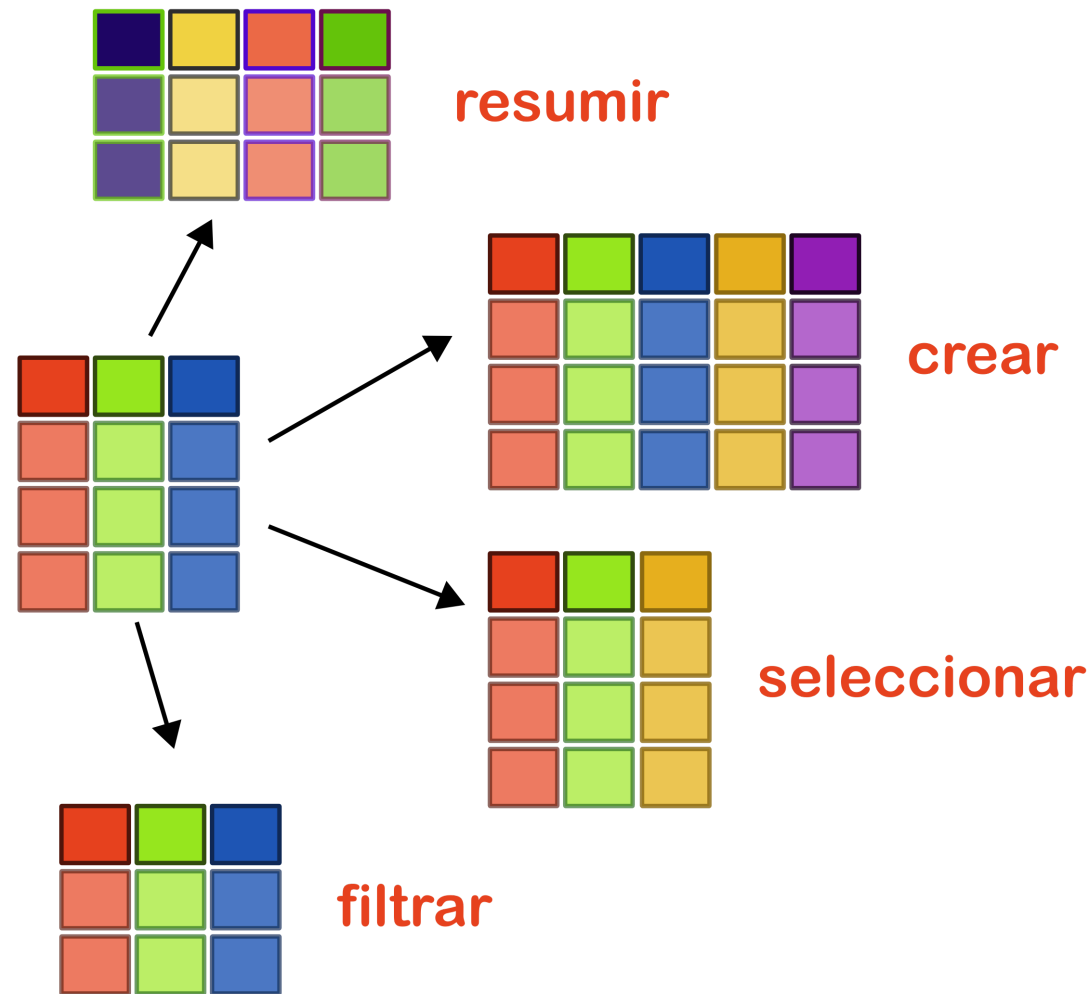
La gran idea: verbos sobre una tabla

Todos los verbos de dplyr siguen la misma forma: **verbo (datos, qué hacer)** . El primer argumento siempre es el data frame.

Verbo	Qué hace	Equivalente R base (Semana 4)
<code>select()</code>	elige columnas	<code>df[, c("a", "b")]</code>
<code>filter()</code>	elige filas	<code>df[df\$x > 0,]</code>
<code>mutate()</code>	crea columnas	<code>df\$nueva <- ...</code>
<code>summarise()</code>	colapsa a un resumen	<code>mean(df\$col)</code>
<code>arrange()</code>	ordena filas	<code>df[order(df\$x),]</code>

Si entienden esta tabla, entienden dplyr. El resto es práctica.

Los cuatro verbos, en un diagrama



Cada verbo cambia la tabla de una forma distinta: **filtrar** quita filas · **seleccionar** quita columnas · **crear** agrega una columna · **resumir** colapsa todo a menos filas.

`select()` — elegir columnas

Quedarse con las columnas que importan

```
# R base (Semana 4)
trabajadores[ , c("region", "ingreso")]
```

```
  region ingreso
1  Ñuble  450000
2 Biobío  920000
3  Ñuble  380000
4  Maule  510000
```

```
# dplyr: sin comillas, sin c(), sin [ , ]
select(trabajadores, region, ingreso)
```

```
  region ingreso
1  Ñuble  450000
2 Biobío  920000
3  Ñuble  380000
4  Maule  510000
```

También: `select(trabajadores, -genero)` **quita** una columna. Y cuando las columnas son muchas, se pueden elegir por **patrón** — eso viene en dos slides más.

Por qué importa en datos reales

CASEN trae **cientos** de columnas. Lo primero al abrir un dataset grande es `select()` las que se van a usar:

```
casen <- read.csv("../data/raw/casen_reducido.csv")  
head(select(casen, region, sector, ingreso), 4)
```

	region	sector	ingreso
1	Ñuble	Agricultura	520000
2	Metropolitana	Agricultura	467000
3	Metropolitana	Industria	NA
4	Metropolitana	Construcción	363000

Por ahora las funciones se **anidan**: `head()` envuelve a `select()`. Al final de la sesión veremos cómo desenredar esto.

Una base con el ingreso desagregado

`casen_ingresos.csv` tiene **las mismas 60 personas**, pero el ingreso abierto por fuente — como viene la CAsEN de verdad:

```
ingresos <- read.csv("../data/raw/casen_ingresos.csv")
names(ingresos)
```

```
[1] "region"      "sector"      "genero"      "educ"        "edad"        "horas"
[7] "ing_trabajo" "ing_capital" "ing_subsidios" "ing_total"
```

Cuatro columnas empiezan con `ing_`. Escribirlas a mano es tedioso y frágil: si mañana llega `ing_arriendo`, hay que acordarse de agregarla. Para eso existen los **selectores por patrón**.

starts_with() : elegir por nombre

```
head(select(ingresos, starts_with("ing")), 3)
```

	ing_trabajo	ing_capital	ing_subsidios	ing_total
1	470000	0	50000	520000
2	434000	0	33000	467000
3	NA	NA	NA	NA

```
# La familia completa: por prefijo, por sufijo, por contenido  
names(select(ingresos, ends_with("total")))
```

```
[1] "ing_total"
```

```
names(select(ingresos, contains("sub")))
```

```
[1] "ing_subsidios"
```

La ventaja no es escribir menos: es que el código **sigue siendo correcto** si mañana el dataset trae una columna `ing_` más. Se elige por regla, no por lista.

where () : elegir por tipo

`starts_with()` mira el **nombre**. `where()` mira el **contenido** de la columna:

```
names(select(ingresos, where(is.numeric)))
```

```
[1] "educ"          "edad"          "horas"          "ing_trabajo"    "ing_capital"
"ing_subsidios"
[7] "ing_total"
```

```
names(select(ingresos, where(is.character)))
```

```
[1] "region" "sector" "genero"
```

Los dos selectores **no dan lo mismo**: `starts_with("ing")` devuelve 4 columnas; `where(is.numeric)` devuelve 7, porque `educ`, `edad` y `horas` también son números. Elegir el selector equivocado es un error silencioso: no falla, calcula sobre columnas que no querías.

`where(is.numeric)` es lo primero que se escribe al calcular descriptivos: nadie quiere el promedio de la columna `region`.

`filter()` — elegir filas

Filtrar filas por condición

```
# R base (Semana 4): repetir el nombre del data frame, $ por todos lados
trabajadores[trabajadores$edad > 30 & trabajadores$educ >= 12, ]
```

	region	sector	educ	edad	ingreso	genero
1	Ñuble	Agricultura	12	34	450000	M
2	Biobío	Servicios	16	45	920000	F

```
# dplyr: la columna se nombra directo; la coma equivale a Y (&)
filter(trabajadores, edad > 30, educ >= 12)
```

	region	sector	educ	edad	ingreso	genero
1	Ñuble	Agricultura	12	34	450000	M
2	Biobío	Servicios	16	45	920000	F

Dentro de `filter()` se escribe `edad`, no `trabajadores$edad`. La **coma** = `&` (Y); para "O" se usa `|`.

Comparaciones útiles

```
filter(trabajadores, region == "Ñuble") # igualdad exacta
```

	region	sector	educ	edad	ingreso	genero
1	Ñuble	Agricultura	12	34	450000	M
2	Ñuble	Comercio	8	52	380000	M

```
filter(trabajadores, region %in% c("Ñuble", "Maule")) # pertenencia
```

	region	sector	educ	edad	ingreso	genero
1	Ñuble	Agricultura	12	34	450000	M
2	Ñuble	Comercio	8	52	380000	M
3	Maule	Agricultura	12	29	510000	F

```
nrow(filter(casen, !is.na(ingreso))) # quitar NA (clave para la T2)
```

```
[1] 55
```

== compara, = asigna. Confundirlos es el error #1 del principiante.

Combinar condiciones: & y |

```
# & = Y -> se deben cumplir LAS DOS  
nrow(filter(ingresos, region == "Ñuble" & educ >= 16))
```

```
[1] 3
```

```
# la barra vertical = O -> basta con que se cumpla UNA  
nrow(filter(ingresos, region == "Ñuble" | region == "Biobío"))
```

```
[1] 25
```

La **coma** dentro de `filter()` es exactamente un `&`. Se escribe `&` cuando la condición se mezcla con un `|`, porque ahí la coma ya no alcanza.

Regla para leerlas: `&` **achica** el resultado (pide más), `|` lo **agrand**a (se conforma con menos).

El paréntesis que cambia la respuesta

```
# CON paréntesis: (Ñuble o Biobío) Y educación sobre 12  
nrow(filter(ingresos, (region == "Ñuble" | region == "Biobío") & educ > 12))
```

```
[1] 10
```

```
# SIN paréntesis: Ñuble entero, O los de Biobío con educación sobre 12  
nrow(filter(ingresos, region == "Ñuble" | region == "Biobío" & educ > 12))
```

```
[1] 20
```

10 contra 20: el doble. `&` se evalúa **antes** que `|`, igual que la multiplicación antes que la suma. Sin paréntesis R no da error — devuelve otra pregunta, respondida correctamente.

Por eso: **cuando mezclen `&` con `|`, pongan paréntesis siempre**, aunque crean que no hacen falta. Es el error que más veces pasa inadvertido en un informe.

`mutate()` — crear / modificar columnas

Crear columnas derivadas

```
# R base: trabajadores$ingreso_miles <- trabajadores$ingreso / 1000
# dplyr: varias a la vez, y una puede usar a la anterior
mutate(trabajadores,
       ingreso_miles = ingreso / 1000,
       log_ingreso   = round(log(ingreso), 2))
```

	region	sector	educ	edad	ingreso	genero	ingreso_miles	log_ingreso
1	Ñuble	Agricultura	12	34	450000	M	450	13.02
2	Biobío	Servicios	16	45	920000	F	920	13.73
3	Ñuble	Comercio	8	52	380000	M	380	12.85
4	Maule	Agricultura	12	29	510000	F	510	13.14

Variables que se construyen SIEMPRE en economía

Esto es exactamente lo que pedirá la T2:

```
mutate(trabajadores,  
  pobre = ingreso < 216000, # binaria (lógica)  
  grupo_etario = if_else(edad < 30, "joven", "adulto")) # categoría condicional
```

	region	sector	educ	edad	ingreso	genero	pobre	grupo_etario
1	Ñuble	Agricultura	12	34	450000	M	FALSE	adulto
2	Biobío	Servicios	16	45	920000	F	FALSE	adulto
3	Ñuble	Comercio	8	52	380000	M	FALSE	adulto
4	Maule	Agricultura	12	29	510000	F	FALSE	joven

`mutate() + if_else()` es **cómo se construyen los indicadores** (pobreza, tramos de edad, dummies) en el trabajo real. `if_else()` (dplyr) es más estricto que `ifelse()` (base): preferirlo.

Cuando las categorías son más de dos

`if_else()` solo sabe decir **sí o no**. Para tres o más niveles se usa `case_when()`: una lista de `condición ~ valor`.

```
clasificados <- mutate(trabajadores,  
  nivel = case_when(  
    educ < 12 ~ "Sin media completa",  
    educ == 12 ~ "Media completa",  
    educ > 12 ~ "Superior"  
  ))  
select(clasificados, educ, ingreso, nivel)
```

	educ	ingreso		nivel
1	12	450000	Media	completa
2	16	920000		Superior
3	8	380000	Sin media	completa
4	12	510000	Media	completa

Se evalúa **de arriba hacia abajo** y gana la primera condición que se cumple. Por eso el orden importa: si la primera fuera `educ > 0`, todas las filas caerían ahí.

case_when() : el caso por defecto

Si ninguna condición se cumple, la fila queda en `NA`. Para evitarlo se cierra con `TRUE ~ ...`, que significa “todo lo demás”:

```
tramos <- mutate(trabajadores,  
  tramo = case_when(  
    edad < 30 ~ "joven",  
    edad < 45 ~ "adulto",  
    TRUE     ~ "mayor"    # todo lo que no calzó arriba  
  )  
)  
select(tramos, edad, tramo)
```

```
edad  tramo  
1    34 adulto  
2    45 mayor  
3    52 mayor  
4    29 joven
```

Fíjense que la segunda condición dice `edad < 45` a secas, no `edad >= 30 & edad < 45`: los menores de 30 **ya salieron** en la línea anterior. Es lo que hace a `case_when()` legible.

Tramos de edad, niveles educativos, quintiles, categorías de pobreza: en economía casi ninguna clasificación tiene solo dos casos. `case_when()` es la herramienta para todas ellas — y la van a necesitar en la T2.

`summarise()` — resumir

Colapsar la tabla a un resumen

```
# R base: valores sueltos. dplyr: una tabla de resumen con nombres
summarise(trabajadores,
  ingreso_medio = mean(ingreso),
  ingreso_mediano = median(ingreso),
  n = n()) # n() cuenta las filas
```

```
  ingreso_medio ingreso_mediano n
1         565000         480000 4
```

`mutate` **mantiene** las filas (agrega columna); `summarise` **colapsa** todo a una fila. `n()` es propia de dplyr: cuenta observaciones – aparecerá en toda la T2.

💬 ¿Y si quiero el ingreso medio *por sector*, no del total? → eso es `group_by()`, la próxima sesión.

El pipe — encadenar

El problema que resuelve el pipe

```
# Anidado: se lee de adentro hacia afuera 😞
summarise(filter(select(trabajadores, sector, ingreso), ingreso > 400000),
           medio = mean(ingreso))

# Con objetos intermedios: funciona, pero llena el environment de basura
t1 <- select(trabajadores, sector, ingreso)
t2 <- filter(t1, ingreso > 400000)
summarise(t2, medio = mean(ingreso))
```

El pipe `|>`: “y luego...”

Toma el resultado de la izquierda y lo pasa como **primer argumento** de la derecha. Se lee de arriba hacia abajo, como una receta:

```
trabajadores |>
  select(sector, ingreso) |>
  filter(ingreso > 400000) |>
  summarise(medio = mean(ingreso))
```

```
      medio
1 626666.7
```

“Toma `trabajadores` , **y luego** elige columnas, **y luego** filtra, **y luego** resume.” En este curso usamos `|>` (nativo de R); en código ajeno verán `%>%` — son casi equivalentes. Atajo RStudio: `Ctrl/Cmd + Shift + M`.



IMAGEN: tubería / línea de ensamblaje: el dato entra por un extremo y sale procesado.

El flujo completo de un análisis

Los cinco verbos en un solo pipe que se lee solo:

```
trabajadores |>
  filter(!is.na(ingreso)) |>
  mutate(ingreso_miles = ingreso / 1000) |>
  select(region, sector, ingreso_miles) |>
  arrange(desc(ingreso_miles))          # ordenar de mayor a menor (Sesión 2)
```

	region	sector	ingreso_miles
1	Biobío	Servicios	920
2	Maule	Agricultura	510
3	Ñuble	Agricultura	450
4	Ñuble	Comercio	380

Esto es lo que escribe un economista todos los días. Y se lee como una frase.

Cierre: lo que nos llevamos hoy

Verbo	Hace	Recordatorio
<code>select()</code>	elige columnas	sin comillas
<code>filter()</code>	elige filas	coma = Y; == no =
<code>mutate()</code>	crea columnas	+ <code>if_else()</code> si son 2 casos
<code>case_when()</code>	clasifica en 3+	gana la primera condición; cerrar con <code>TRUE</code>
<code>summarise()</code>	colapsa a resumen	+ <code>n()</code> para contar
<code>\ ></code>	encadena	"y luego..."

En una línea: **dplyr** es leer el análisis como una frase.

Puente a la Sesión 2: hoy resumimos el total; ahora lo más potente — resumir **por grupos** con `group_by()` : ingreso por sector, región y género. Y el **laboratorio prepara la T2 (calificada, 13 %)**.



Sesión 1 — Fin

Break de 10–15 minutos

A continuación: **Sesión 2 — group_by y Aplicaciones**

`group_by()` · `arrange()` · workflow completo